

VTEX Golden Path com GenAI

Como a VTEX usa Spec-Driven Development com Claude Code



Claude Impact Lab · Rio de Janeiro · 2026

Lightning Talk | Claude Impact Lab



Lucas Schlee

Staff Software Engineer · VTEX



Por que SDD?

O que a IA quebrou no SDLC tradicional

Capítulo

01

IA gera código rápido.
Mas de fato o tempo de
lançamento está sendo
acelerado?

O SDLC com IA está quebrado

4 sintomas que todo time conhece

01

Specs desconectadas

Documento de requisitos escrito por fora. IA não sabe da existência. O código vai por outro caminho.

02

PRs gigantes sem intenção

Diff de 2.000 linhas. Review impossível. Revisor aprova confiando — não validando.

03

Contexto vira fofoca

Decisões estratégicas de priorização ficam em threads de Slack. Próximo dev (ou IA) não acha.

04

Review é gargalo

IA escreveu em 5 min. Review demora 2 dias. O ganho de velocidade evapora.

A escala VTEX

Por que isso
é um problema sistêmico

200+

engenheiros

3.7k+

repositórios

Sem governança, 200 devs negociam com a IA de 200 formas diferentes.

Por que SDD funciona

O contrato executável, em ação

SEM SDD



COM SDD



Plataforma de saúde: Próxima visita

Do zero ao MVP em 2 horas

A plataforma em ação

Plataforma para Agentes Comunitários de Saúde: lista priorizada por risco + mapa do bairro, tudo em uma tela.



Código fonte

[github.com/lischlee/
poc-sdd-claude-impact](https://github.com/lischlee/poc-sdd-claude-impact)

Fila de Visitas

Fila de Visitas

Família Ferreira

0.85

Nunca visitada

365 dias sem visita
2 residente(s) vulnerável(eis)
4 condição(ões) crônica(s)

[Registrar visita →](#)

Família Martins

0.73

Nunca visitada

365 dias sem visita
4 condição(ões) crônica(s)
2 residente(s) vulnerável(eis)

[Registrar visita →](#)

Família Mendes

0.73

Nunca visitada

365 dias sem visita
4 condição(ões) crônica(s)
1 residente(s) vulnerável(eis)

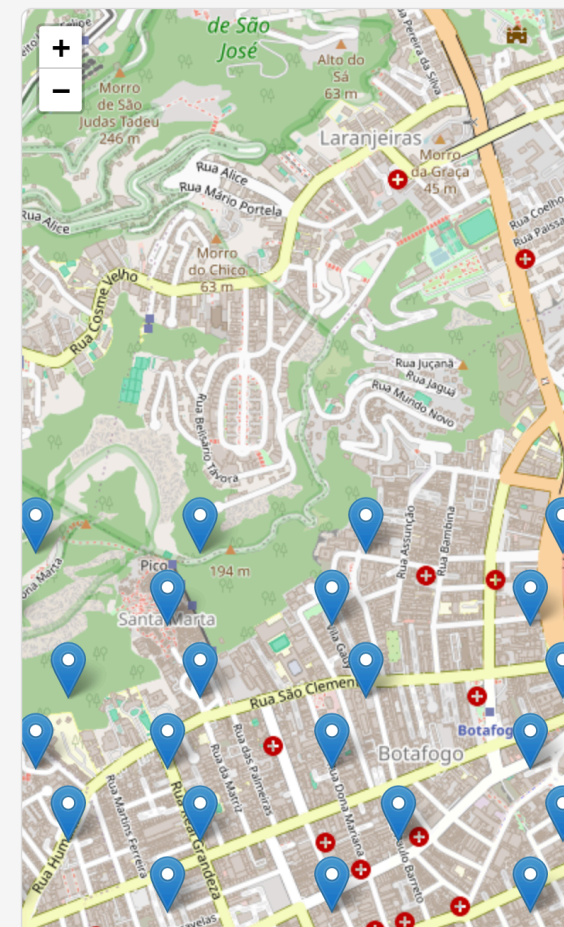
[Registrar visita →](#)

Família Oliveira

0.68

Nunca visitada

365 dias sem visita
3 condição(ões) crônica(s)
2 residente(s) vulnerável(eis)



Agente seleciona a próxima família

Toca em "Registrar visita" da Família Ferreira — a de maior risco no dia (score 0.85).

Fila de Visitas

Fila de Visitas

Família Ferreira

0.85

Nunca visitada

365 dias sem visita

2 residente(s) vulnerável(eis)

4 condição(ões) crônica(s)

Registrar visita →

Família Martins

0.73

Nunca visitada

365 dias sem visita

4 condição(ões) crônica(s)

2 residente(s) vulnerável(eis)

Registrar visita →

Família Mendes

0.73

Nunca visitada

365 dias sem visita

4 condição(ões) crônica(s)

1 residente(s) vulnerável(eis)

Registrar visita →

Família Oliveira

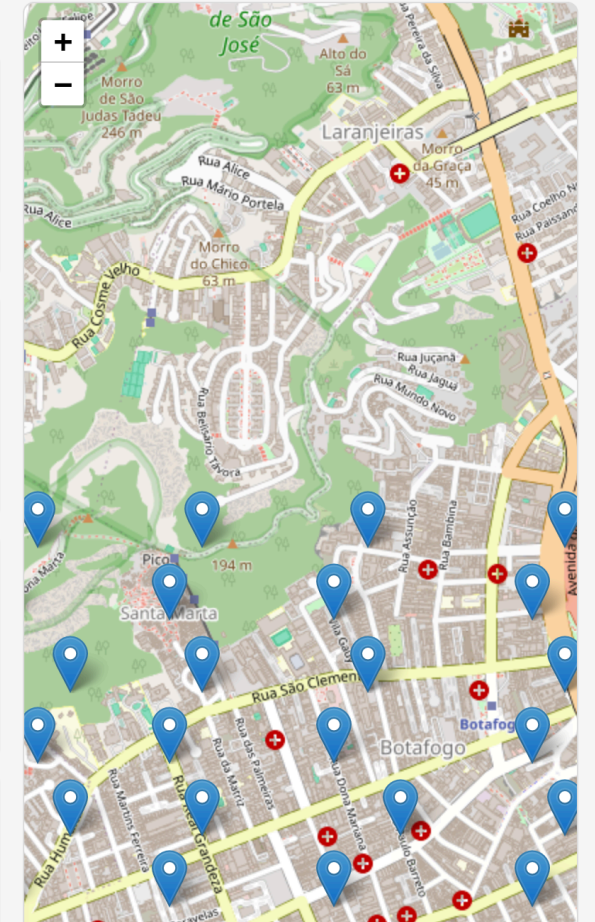
0.68

Nunca visitada

365 dias sem visita

3 condição(ões) crônica(s)

2 residente(s) vulnerável(eis)



Formulário em campo

Data, observações, ações de acompanhamento. 30 segundos end-to-end (SC-002).

[← Voltar à fila](#)

Família Ferreira

Residentes

Idoso (diabetes, doença cardíaca, hipertensão)

Idoso (artrite)

Registrar Visita

Data da visita

05 / 22 / 2026



Observações

Família visitada. Pressão arterial aferida (12/8). Encaminhamento agendado para gestante.

Ações de acompanhamento

- ☐ Pressão arterial não aferida
- ☐ Encaminhamento necessário
- ☐ Revisão de medicação necessária
- ☐ Retorno agendado

Registrar

A fila se reordena

Família Ferreira saiu do topo. Martins assumiu (score 0.73). Recalc em < 3s (SC-003).

Fila de Visitas

Fila de Visitas

Família Martins

0.73

Nunca visitada

365 dias sem visita
4 condição(ões) crônica(s)
2 residente(s) vulnerável(eis)

Registrar visita →

Família Mendes

0.73

Nunca visitada

365 dias sem visita
4 condição(ões) crônica(s)
1 residente(s) vulnerável(eis)

Registrar visita →

Família Oliveira

0.68

Nunca visitada

365 dias sem visita
3 condição(ões) crônica(s)
2 residente(s) vulnerável(eis)

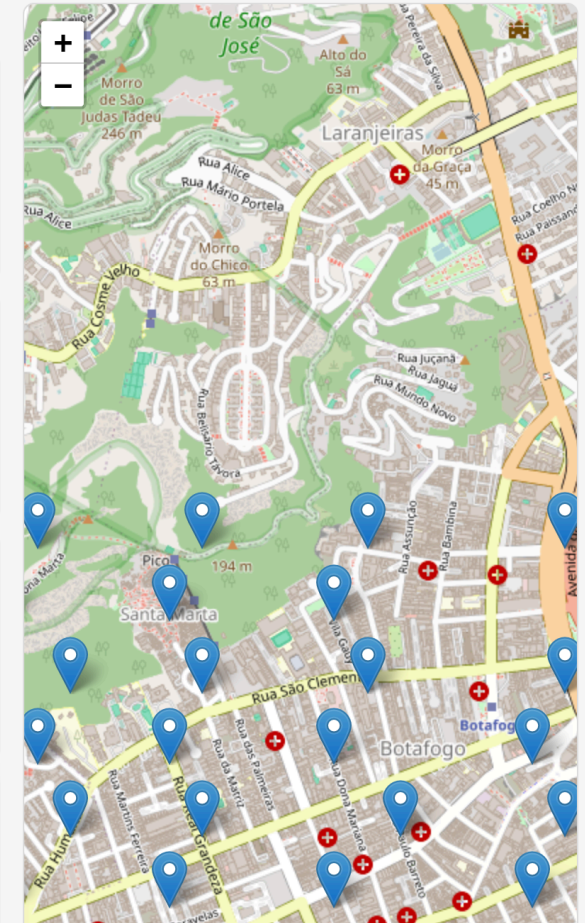
Registrar visita →

Família Nascimento

0.68

Nunca visitada

365 dias sem visita
3 condição(ões) crônica(s)
2 residente(s) vulnerável(eis)



Do zero ao MVP

Em 2 horas de pair-programming com Claude Code

2h

do zero ao MVP
demonstrável

41

tasks executadas
(100% concluídas)

13

testes E2E
passando

0

retrabalho de
escopo

spec-kit, fase por fase

03

Constitution → Specify → Clarify → Plan → Implement

O fluxo Spec Kit

4 fases. Cada uma é um gate.



1 • Constitution

Princípios não-negociáveis. Os guardrails que toda IA precisa respeitar.



2 • Specify + Clarify

Intenção em linguagem natural. IA pergunta o que falta. Specs ficam testáveis.



3 • Plan + Tasks + Analyze

Decomposição em fatias verticais com QA cruzado entre spec, plan e tasks.



4 • Implement

Execução task-by-task. Commit a cada verde. Constitution Check em cada PR.

Fase 1 | Spec Kit

/speckit-
constitution

3.1

O prompt (literal)

Linguagem natural vira guardrails

/speckit-constitution this project should follow the test driven design. for every change you do, make sure to test it first, see it failing, than proceed.

when building features, follow the vertical slice approach: build each increment vertically, so you can validate the whole stack in every feature increment.

→ constitution.md v1.0.0 (TDD + Vertical Slice, 5 quality gates).

Constitution

Core Principles

I. Test-Driven Development (NON-NEGOTIABLE)

Every change — feature work, bug fix, or refactor that alters observable behavior — **MUST** begin with an automated test that fails for the right reason before any production code is written or modified. The cycle is strictly Red → Green → Refactor:

- **Red:** Author the test first. Run it. Confirm it fails, and confirm the failure message points at the missing or incorrect behavior (not at a setup mistake, a typo, or a missing import).
- **Green:** Write the minimum production code required to make the test pass. No speculative code, no untested branches.
- **Refactor:** With tests green, improve structure without changing behavior. Tests must remain green after every refactor step.

Pure refactors that preserve behavior **MAY** proceed under existing test coverage, provided that coverage demonstrably exercises the code paths being changed. If the affected paths are uncovered, a characterization test **MUST** be added first.

Rationale: Writing the test first forces the change to be specified in terms of observable behavior before implementation bias sets in. Seeing the test fail proves the test actually validates the new behavior rather than passing vacuously. This is the discipline that lets the rest of the constitution — vertical slices, quality gates, stay honest

Fase 2 | Spec Kit

/speckit-specify

3.2

O prompt seminal (literal)

3 linhas viraram uma spec de 140 linhas

```
/speckit-specify  
app de "próxima visita":  
tela mostrando mapa de bairro + lista  
de família do dia ordenadas por risco  
score combinando tempo desde última  
visita, condições crônicas,  
idosos/gestantes/bebês, etc  
agente de saúde registra visita,  
score recalcula, lista muda
```

→ 2 user stories testáveis · 18
functional requirements · 6
success criteria · 8 perguntas
críticas levantadas.

User Story 1 — See today's prioritized visit queue on map + list (Priority: P1)

A community health agent (CHA) opens the app at the start of their shift and immediately sees the families they should visit today, ranked from highest to lowest risk. The same families are pinned on a neighborhood map so the agent can plan a walking route. Each list entry shows the family identifier, a clear risk indicator, and the main drivers of that risk (e.g., "no visit in 38 days", "1 pregnant resident", "1 elderly with hypertension").

Why this priority: This is the smallest end-to-end slice that delivers the app's core value — replacing a paper checklist or memory-based prioritization with an explicit, risk-aware queue. Without this slice the product has no reason to exist; with this slice alone (even read-only) the agent already gets daily decision support.

Independent Test: With a seeded set of families in one micro-area (last-visit dates, household composition, chronic conditions), launch the app, sign in as the CHA assigned to that micro-area, and confirm: (a) the list shows N families in descending risk order, (b) the map shows the same N families as pins matching the list ranking, (c) tapping a list entry highlights the matching map pin and vice versa, and (d) the displayed risk drivers match the seeded data.

Acceptance Scenarios:

1. **Given** a CHA assigned to a micro-area with 12 enrolled families, and 8 of those families flagged for visit today, **When** the CHA opens the app, **Then** the home screen shows a map of the micro-

Fase 2 | Spec Kit

/speckit-clarify

3.3

Clarifications geradas no spec.md

A IA pergunta o que falta

6 decisões de escopo respondidas antes do plan:

1. Auth → sem login; identidade fixa no build
2. Conectividade → zero rede em campo
3. PHI → plaintext aceito como risco do POC
4. Volume → ~30 famílias, ~50 visitas no piloto
5. Idioma → pt-BR (sem toggle de locale)
6. Export → supervisor lê via DevTools (sem UI)

Escopo travado upfront — sem NEEDS

CLARIFICATION pendentes.

Clarifications

Session 2026-05-21

- Q: How are community health agents identified by the app? → A: No interactive auth in the POC; a single CHA identity is baked into the pilot build and used for all audit-logged visit registrations.
- Q: How offline-capable must the MVP be? → A: No network at all in the POC — mocked roster ships in the build, visits written to on-device storage, no sync or remote backend calls in scope; off-device export is manual and out of scope for the in-app UX.
- Q: What is the target client platform for the POC? → A: Next.js web app with TypeScript across the full stack; accessed via mobile browser on the field device.
- Q: How is PHI (chronic conditions, pregnancy status, addresses) protected at rest in browser storage? → A: Plaintext in browser storage — POC accepts the risk; only the pilot device handles this data and it is wiped after pilot. Documented as a known POC limitation.
- Q: What is the target data volume for the pilot? → A: ~30 families, ~3 residents each, ~50 total visits over pilot — minimal demo fixture; map shows all pins unclustered.
- Q: What UI language(s) does the MVP support? → A: pt-BR only for the MVP, with all user-facing strings routed through a thin i18n layer so future languages are easy to add.
- Q: How are map tiles delivered under the no-network constraint? → A: Bundle a pre-generated static raster tile pack (OpenStreetMap tiles for the pilot bounding box at 2–3 zoom levels) inside the Next.js build, served from `/public/`. True zero-network.
- Q: How does visit data leave the pilot device for supervisor review (SC-005)? → A: No in-app export. Supervisor uses browser DevTools / IndexedDB inspector to dump data manually at pilot

/speckit-plan
/speckit-tasks
/speckit-analyze

3.4

tasks.md gerado pela IA

41 tasks em 4 fases

Phase 1: Setup (Shared Infrastructure)

- [X] T001 Initialize Next.js 14 App Router project...
- [X] T002 Install dependencies: react-leaflet, idb, next-intl...
- [X] T004 [P] Configure Playwright in playwright.config.ts...

Phase 2: Foundational (Blocking Prerequisites)

- [X] T009 Define shared TypeScript types in src/domain/models.ts
- [X] T012 Define IndexedDB schema in src/lib/storage/db.ts

Phase 3: User Story 1 – Prioritized queue (MVP)

- [X] T014 [P] [US1] Write FAILING Playwright E2E test FIRST
- [X] T018 [US1] Implement computeRiskScore (makes T015 green)
- [X] T024 [P] [US1] Implement QueueMap with offline tiles

Phase 4: User Story 2 – Register visit + re-prioritize

- [X] T027 [P] [US2] Write FAILING Playwright E2E test FIRST
- [X] T029 [US2] Implement VisitRepository (IndexedDB)

Fase 4 | Spec Kit

/speckit-
implement

3.5

A história, em commits

git log auditável

```
0fc2e44 feat(setup): initialize Next.js 14 project (Phase 1)
f0c6dd9 feat(foundation): domain models, roster, IndexedDB (Phase 2)
2891059 test(us1): write FAILING tests for scoring, queue, E2E
72004f4 feat(domain): implement scoring, queue sort+filter (Red→Green)
8552d7e feat(ui/us1): useVisitQueue, FamilyCard, QueueList, QueueMap
e102aa6 feat(config): switch playwright firefox (T026)
3e0f2e4 test(us2): write FAILING tests for visit registration
8896dd7 feat(us2): VisitRepository, VisitService, VisitForm
9d00f69 feat(e2e): all 13 E2E tests passing – stories 1 & 2 complete
```

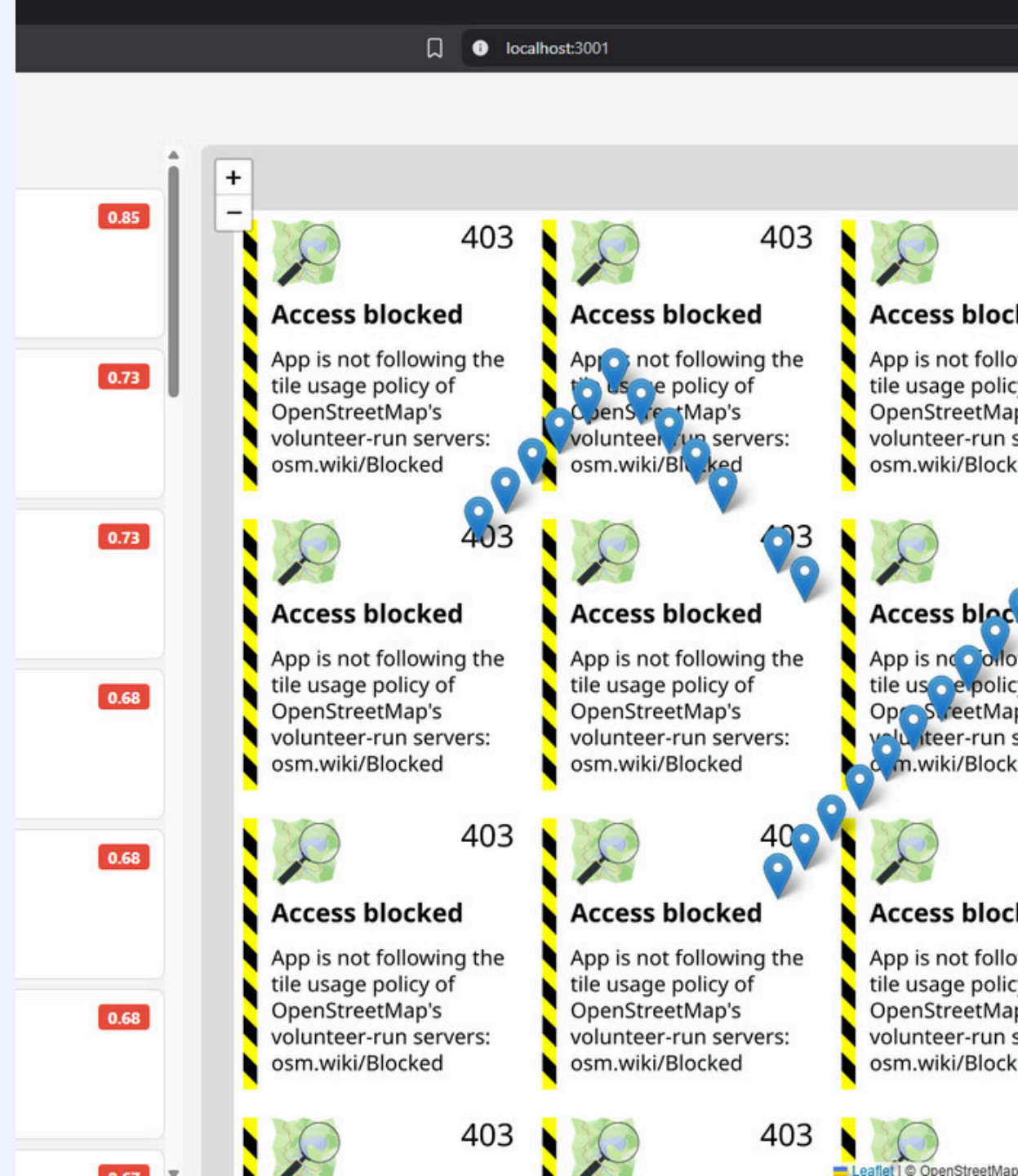
Cada commit reflete uma task atômica do `tasks.md`. História auditável: o reviewer vê a ordem Red → Green declarada na constitution.

Iteração em campo

Mas o mundo real reage.

13 E2E verdes. Mas no browser, 3 bugs que os testes não pegaram:

- Labels i18n quebrados
- Mapa em branco (404 nos tiles)
- Tiles bloqueados pelo OSM (403)



3 prompts conversacionais. 1 commit fix.

- 13:01** → "I've had a couple of errors when running manually after a pnpm dev. help me fix it" + *screenshot DevTools*
- 13:14** → "the map is still not showing. maybe related to the 404 errors?" + *screenshot*
- 13:31** → "the map is loading the tiles, but they show this 403 error. link to the doc: osm.wiki/Blocked" + *2 screenshots*

```
f1d5a2d fix(map): fix Leaflet icons, OSM tile  
source, and risk factor i18n
```

Capítulo

Human- in-the- loop

SDD estrutura. Revisão garante.

04

Onde o humano fecha o loop

Na Constitution

Quais princípios você marca non-negotiable moldam toda IA daqui pra frente. Não delegue.

Na Spec

Escopo. Edge cases. Decisões de negócio. SDD não decide isso por você — só estrutura o que você decidir.

Na produção

Quem responde ao incidente às 3 da manhã continua sendo humano. PR pequeno é revisável; diff gigante, não — manter a slice estreita é estratégia de operação.

Obrigado! |

Lucas Schlee
lucas.fernandes@vtex.com



The Composable
and Complete
Commerce Platform